

Car Rental - Visao Tecnica

Data: 21/11/2025

1. Visao Geral do Repositorio

- Projeto full stack de locacao de veiculos com Django 5, Django REST Framework e PostgreSQL.
- Orquestracao via Docker Compose com servicos para backend, banco e frontend Gatsby.
- Estrutura pronta para autenticacao baseada no modelo personalizado Customer (extends AbstractUser).

2. Backend Django

Arquitetura

- App principal `rental` concentra models, forms, views e API.
- Configuracoes em `config/settings.py`, com suporte a variaveis .env, Postgres e fallback SQLite.

Modelagem

- Car: identifica veiculos com placa unica, diaria configuravel e flag de disponibilidade.
- Customer: usuario autenticavel com campos extras (documento, telefone, etc.).
- Rental: reserva vinculando carro e cliente, controla status (reservada, em curso, finalizada, cancelada) e calcula preco total evitando sobreposicao de datas.

API REST

- DRF com paginacao padrao e autenticacao de sessao (SessionAuthentication).
- Viewsets: CarViewSet, CustomerViewSet (apenas staff), RentalViewSet com escopo por usuario e acoes auxiliares (my).
- Serializers expoe detalhes aninhados (car_detail) e protegem campos calculados.
- Endpoints adicionais de sessao: /api/session/, /api/session/login/, /api/session/logout/.

Infraestrutura

- compose/local/Dockerfile monta o servico web com Python 3.12, aplica migracoes e cria superusuario.
- docker-compose.yml define servicos web, db e gatsby, compartilha volumes e expoe portas 8000/8001.
- CORS configurado (django-cors-headers) para permitir consumo pelo Gatsby.
- Script de entrada garante migrations automaticas e criacao de usuario admin para testes.

3. Frontend Django (Templates)

- Renderiza HTML server-side com Pico.css via templates/.
- Painel (/) mostra reservas do usuario e carros ativos com formularios baseados em RentalForm.
- Fluxo de reserva em /rentals/nova/ usa CSRF padrao, validacoes server-side e redireciona via CBVs.
- Autenticacao aproveita as views nativas de django.contrib.auth.

4. Frontend Gatsby (SPA)

Arquitetura

- Projeto React/Gatsby em frontend/ com build Node 20 e hot reload (porta 8001).
- Contexto global de autenticacao (AuthContext) conversa com os endpoints de sessao (cookies).
- Cliente generico api/client.js centraliza fetch com CSRF e credenciais.
- Paginas principais: dashboard (index.js), login (login.js), criacao de reserva (rentals/nova.js).

UX e Dados

- Dashboard consome `/api/cars/` e `/api/rentals/my/`, exibe resumo, tabela de reservas e cartões de carros.
- Formulário SPA replica campos do Django, preenche diário automaticamente e trata erros da API em linha.
- Navegação client-side com Link, feedback visual via alerts e componentes reutilizáveis.
- Build preparado para produção (`npm run build`) e integrado ao Docker Compose.

5. Comparativo Gatsby x Django Templates

Renderização

- Django entrega HTML gerado no servidor; Gatsby gera SPA estática com hidratação React.
- Gatsby permite navegação instantânea e mudanças de estado sem reload; Django depende de round-trips.

Consumo de Dados

- Django usa ORM diretamente nas views, contexto passado ao template.
- Gatsby atua como cliente da API REST, respeitando regras de autenticação via cookies.

Autenticação

- Templates usam sessão tradicional com formulários Django e proteção CSRF automática.
- Gatsby reutiliza a mesma sessão com chamadas `/api/session/*`, mantendo cookies e estado no contexto.

Formulários

- Django depende de `RentalForm` para validação server-side e rendering automático.
- Gatsby implementa formulário controlado em React, exibindo mensagens de erro retornadas pela API.

Build e Deploy

- Django roda via Unicorn/Daphne dentro do container web, servindo templates.
- Gatsby constrói assets estáticos (ou roda `gatsby develop`) em container Node separado, consumindo a API.

6. Próximos Passos Sugeridos

- Automatizar limpeza de cache do Gatsby para evitar problemas de permissão em builds locais.
- Adicionar testes end-to-end cobrindo o fluxo SPA (Playwright ou Cypress).
- Avaliar GraphQL do Gatsby ou bibliotecas de cache cliente (SWR/React Query).
- Implementar internacionalização compartilhada entre os dois frontends.